

Wallabot Documentation

Wallabot is an AI-powered C2C second-hand marketplace built with a FastAPI microservices backend, a Vue 3 frontend, and an LLM-driven agentic subsystem for smart product classification and price recommendations.

Project Overview

- [Project Overview](#)
 - [What is Wallabot?](#)
 - [Key Features](#)
 - [Technology Stack](#)
 - [Repository Layout](#)
 - [Local Quick-Start](#)
- [System Architecture](#)
 - [Microservices Overview](#)
 - [Service Responsibilities](#)
 - [Authentication Flow](#)
 - [Product State Machine](#)
 - [Database Strategy](#)

REST API Reference

- [REST API Reference](#)
 - [Auth Service API](#)
 - [Authentication](#)
 - [Health Check](#)
 - [Registration & Login](#)
 - [Social Authentication \(OAuth\)](#)
 - [User Profiles](#)
 - [Ratings](#)
 - [Inventory Service API](#)
 - [Health Check](#)
 - [Products](#)
 - [Image Uploads](#)
 - [ProductOut Schema](#)

Project Overview

What is Wallabot?

Wallabot is a full-stack **C2C (consumer-to-consumer) second-hand marketplace** inspired by Wallapop. It allows users to list products, browse listings, reserve items, and complete purchases through a digital wallet. The platform is augmented by an AI subsystem – the *Wallabot Agentic Service* – which provides intelligent product classification and price recommendations powered by LangChain and OpenAI GPT-4o-mini.

Key Features

- **User authentication** – email/password registration and login with JWT, plus OAuth via Google and Facebook.
- **Product catalogue** – create, browse, filter, and search listings with image uploads.
- **Smart AI assistance** – automatic category suggestion and market-based price estimation when creating a listing.
- **Wallet & transactions** – deposit funds, reserve products, and complete peer-to-peer purchases atomically.
- **User ratings** – buyers and sellers can rate each other after a completed transaction.
- **Internationalisation** – the frontend supports English and Spanish (`vue-i18n`).

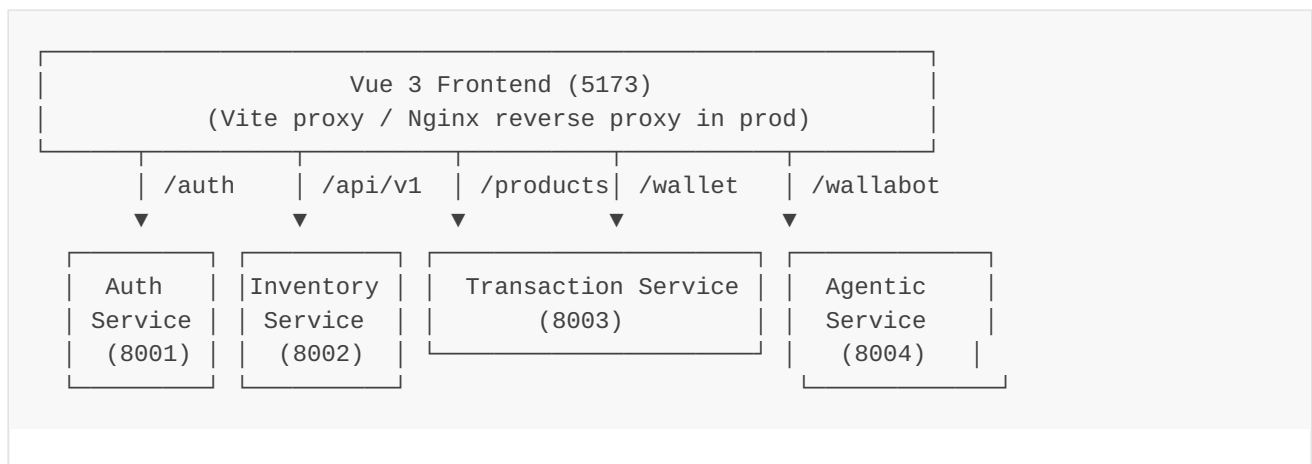
Technology Stack

Layer	Technology
Frontend	Vue 3, Vite, Pinia, Vue Router, Axios, vue-i18n
Backend (all services)	Python 3.11+, FastAPI 0.135, SQLAlchemy 2, Pydantic v2
AI / Agentic subsystem	LangChain LCEL, OpenAI GPT-4o-mini, Tavily Web Search
Observability	LangSmith tracing (optional), structlog structured logging
Database (local)	SQLite per service
Database (production)	PostgreSQL on Supabase, 3 service-scoped roles
Container	Docker, Docker Compose
Deployment	Render Blueprint
CI/CD	GitHub Actions

System Architecture

Microservices Overview

Wallabot follows a **microservices architecture**: each domain is encapsulated in an independently deployable FastAPI service. The Vue 3 frontend acts as the sole entry point for end users and proxies all API calls to the appropriate backend service.



Service Responsibilities

Auth Service (port 8001)

Owns user identity. All other services trust Auth Service-issued JWTs for authentication.

- User registration and login (`/auth/register` , `/auth/login`)
- Social OAuth flow with Google and Facebook (`/auth/social/...`)
- Public user profiles and ratings (`/users/{id}/profile` , `/users/{id}/ratings`)
- JWT access-token issuance with `python-jose` + `bcrypt` password hashing

Inventory Service (port 8002)

Owns the product catalogue. Products here represent the *listing* view.

- Full CRUD for product listings (`/api/v1/products`)
- Image upload to local filesystem or Supabase Storage
- Filtering by state, category, price range, condition, and full-text keyword search

REST API Reference

Wallabot exposes its functionality through four independent HTTP REST services. This section documents every public endpoint, request/response schema, and error contract — the equivalent of an RMI interface specification.

All services accept and return **JSON** (`Content-Type: application/json`). Endpoints that require authentication expect a `Bearer` token in the `Authorization` header issued by the Auth Service.

- [Auth Service API](#)
 - [Authentication](#)
 - [Health Check](#)
 - [Registration & Login](#)
 - [Social Authentication \(OAuth\)](#)
 - [User Profiles](#)
 - [Ratings](#)
- [Inventory Service API](#)
 - [Health Check](#)
 - [Products](#)
 - [Image Uploads](#)
 - [ProductOut Schema](#)
- [Transaction Service API](#)
 - [Health Check](#)
 - [Wallet](#)
 - [Products \(Transaction View\)](#)
 - [Transactions](#)
 - [Error Reference](#)
 - [Reservation Cleanup Daemon](#)
- [Agentic Service API \(Wallabot\)](#)
 - [Health Check](#)
 - [Category Suggestion](#)
 - [Price Recommendation](#)
 - [LangSmith Tracing](#)

Auth Service API

Base URL (local): `http://localhost:8001` Base URL (production): configured via `AUTH_SERVICE_URL` environment variable

The Auth Service owns user identity: registration, login, social OAuth, JWT issuance, public user profiles, and the ratings system.

Authentication

Most endpoints in other services require a JWT access token issued by this service. Pass it as a Bearer token in every request:

```
Authorization: Bearer <access_token>
```

Tokens are signed with `HS256` using the shared `SECRET_KEY` environment variable and contain `{"sub": "<user_email>"}` as the payload subject.

Health Check

GET /health

Returns the service liveness status. Used by Docker healthcheck probes.

Response 200

```
{ "status": "ok" }
```

Inventory Service API

Base URL (local): `http://localhost:8002`

The Inventory Service owns the product catalogue: creating, reading, updating, and deleting listings, plus image uploads. Products stored here represent the *listing* view; the Transaction Service maintains a separate copy for commerce state.

Health Check

GET `/health`

```
{ "status": "ok" }
```

Products

GET `/api/v1/products`

Returns a list of all products, optionally filtered. No authentication required.

Query parameters

Parameter	Type	Description
<code>state</code>	<code>Available</code> <code>Reserved</code> <code>Sold</code>	Filter by availability state
<code>category</code>	string (min 1 char)	Filter by category name (case-insensitive)
<code>min_price</code>	float (≥ 0)	Minimum product price in EUR
<code>max_price</code>	float (≥ 0)	Maximum product price in EUR
<code>condition</code>	<code>New</code> <code>Like New</code> <code>Good</code> <code>Fair</code> <code>Poor</code>	Filter by item condition
<code>q</code>	string (min 1 char)	Full-text keyword search across title and description

Response 200 — array of `ProductOut`

Transaction Service API

Base URL (local): `http://localhost:8003`

The Transaction Service owns the commerce domain. It manages a digital wallet per user, coordinates product reservations (with automatic timeout expiry), and processes purchases via atomic wallet debit operations. It also exposes transaction history.

All endpoints require authentication (`Authorization: Bearer <token>`).

Health Check

GET `/health`

```
{ "status": "ok", "service": "transaction-service" }
```

Wallet

GET `/wallet/balance`

Returns the authenticated user's current wallet balance in EUR.

Response 200

```
{ "balance": 250.00 }
```

POST `/wallet/deposit`

Adds funds to the authenticated user's wallet.

Request body

Agentic Service API (Wallabot)

Base URL (local): `http://localhost:8004`

The Agentic Service is the AI subsystem of Wallabot. It provides two intelligent endpoints that assist sellers when creating a product listing: automatic category classification and market-based price estimation.

All endpoints are **stateless** — each request is self-contained with no memory of prior calls. Authentication is **not required** to call these endpoints.

Health Check

GET `/health`

```
{ "status": "ok" }
```

Category Suggestion

POST `/wallabot/category`

Classifies a product into one of the caller-provided categories using a LangChain LCEL chain backed by **GPT-4o-mini**.

The agent selects the most appropriate existing category. It only proposes a new category name (setting `is_new_category: true`) when none of the provided options is a good fit.

On output validation failure the agent retries up to **3 times** with progressively detailed correction feedback. On LLM provider failure it returns the closest available category with `confidence: 0.0` rather than raising a 5xx error.

Testing & Quality

Overview

Wallabot enforces a **50% minimum code coverage** threshold across all backend services. The CI pipeline fails if any service falls below this threshold.

Test Layers

Unit Tests (per service)

Each service has its own `tests/` directory with pytest-based unit tests. Tests use SQLite databases (created in memory or as temp files) to avoid any external dependencies.

Service	Test directory	Coverage threshold
auth-service	<code>backend/auth-service/tests/</code>	50%
inventory-service	<code>backend/inventory-service/tests/</code>	50%
transaction-service	<code>backend/transaction-service/tests/</code>	50%
agentic-service	<code>backend/agentic-service/tests/</code>	50%

Run a single service's tests:

```
cd backend/auth-service
pytest --cov=app --cov-report=term-missing --cov-fail-under=50
```

Integration Tests

Located in `tests/integration/`, these tests exercise cross-service flows using a live Docker Compose stack.

Key flows covered:

- Full registration → login → product creation → purchase flow
- Product state machine transitions
- Wallet deposit → reservation → purchase atomicity

DevOps & CI/CD

Docker Compose (Local Development)

The `docker-compose.yml` at the project root starts the full stack locally.

```
cp .env.example .env # fill in secrets
docker compose up --build
```

Services and ports:

Service	Internal port	Exposed port
auth-service	8000	8001
inventory-service	8000	8002
transaction-service	8000	8003
agentic-service	8000	8004
frontend	5173	5173

All backend containers share the `wallabot-net` Docker network and the `wallabot-data` named volume that persists SQLite database files.

Environment Variables

Copy `.env.example` to `.env` and fill in:

Variable	Description
<code>SECRET_KEY</code>	JWT signing key (shared by all services)
<code>OPENAI_API_KEY</code>	Required by agentic-service for GPT-4o-mini
<code>TAVILY_API_KEY</code>	Required by agentic-service for web search
<code>SUPABASE_URL</code>	(Optional) Supabase project URL for PostgreSQL + Storage
<code>SUPABASE_SERVICE_ROLE_KEY</code>	(Optional) Supabase service-role key
<code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code>	(Optional) Google OAuth